ALGOPROFESSOR AI R&D SOLUTIONS · HYDERABAD

AlgoLang 1.0

The agent-native programming language for trustworthy AI



Dr. S. Satyanarayana, Ph.D., PDF (AI)

Founder, CEO & Chief Agentic AI Scientist · AlgoProfessor AI R&D Solutions · www.algoprofessor.com

github.com/drssnaiml1-cell/algolang · Apache 2.0 · November 2026

Agenda

01 The problem: AI decisions nobody can check
02 What AlgoLang is, and its design principles
03 The language: agents, flows, policies, types
04 Compiler, runtime and the AlgoVM
05 Governance and Trustworthy AI
06 Uncertainty, evals and evidence
07 Real-time events, Quantum AI, interoperability
08 Comparison with programming languages
09 Comparison with agent frameworks
10 22 enterprise applications
11 Toolchain, roadmap, getting started
12 Where AlgoLang fits, and collaboration

The problem

AI agents now make decisions that matter, and nobody can check them

- Loans, triage, claims, admissions and trades are increasingly decided by LLM agents.
- In today's code an agent is just a function call buried in Python: **no policy, no budget, no reviewer, no record**.
- Regulation now asks for risk classes, human oversight, transparency and logs (e.g. the EU AI Act, India's DPDP Act).
- Teams bolt these on afterwards, inconsistently, and often not at all.

Four questions every AI decision should answer

1. Who was allowed to decide?
2. Why was this the decision?
3. Was it fair and robust?
4. Can an auditor prove what happened?

AlgoLang makes the compiler and runtime answer them, every time.

What is AlgoLang?

AlgoLang is a programming language with **no functions**. Every unit of work is an **agent** with a role, a goal, explicit tools, a model and a policy.

Agent

role · goal · tools · model · policy; returns text, score, list, a typed message, or `uncertain<T>`

Flow

the only other construct: call agents, run them in parallel, branch, emit results

Guarantees

policies, budgets, human review and a signed audit log enforced by compiler and runtime

Designed by Dr. S. Satyanarayana at AlgoProfessor AI R&D Solutions, Hyderabad. Open source (Apache 2.0).

Design principles

- **Agents, not functions.** The unit of computation is an accountable agent.
- **Policies are code.** Budgets, forbidden attributes and reviewers are declared, and checked before anything runs.
- **Uncertainty is a type.** `uncertain<score>` samples several times and carries its confidence.
- **Deterministic guards.** Branch on tool verdicts, not model moods.
- **People stay in charge.** `require human`, four-eyes approvals, escalation queues.
- **Evidence by default.** A hash-chained, signed audit record for every call.
- **Tested like software.** Evals for quality, fairness and robustness in CI.
- **Offline first.** A deterministic mock model, so every program runs and tests without an API key.

The language at a glance

A governed loan decision in 20 lines

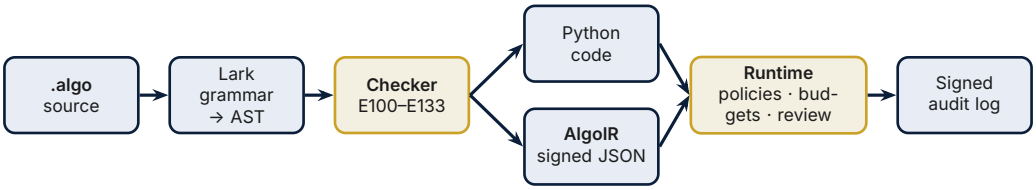
```
risk high "Retail credit decisions"
disclose "An AI system assisted this decision."

agent Affordability(app: text) -> score {
    goal   "Debt-to-income ratio for {app}"
    uses   bureau.pull
    policy { budget tokens 2000  redact pii }
}
agent Committee(app: text, dti: score) -> uncertain<score> {
    goal   "Recommend approval for {app}"
    policy { samples 5  require approvals 2
             forbid gender, religion  explain with evidence }
}
flow main(app: text) {
    dti = Affordability(app)
    if dti > 0.5 { emit Decline(app, dti) }
    else         { emit Committee(app, dti) }
}
```

- `risk high`: the compiler demands oversight, fairness tests, PII redaction and signed audit.
- `budget`, `forbid`: refused at compile time if violated.
- `uncertain<score>`: 5 samples, mean and confidence.
- `require approvals 2`: four-eyes review before release.
- Every call → signed, hash-chained audit.

Compiler and runtime

From source to governed execution



34 compile-time checks: undeclared tools, budgets, forbidden attributes, type errors, unsafe high-risk programs.

Two engines, same result: compiled Python or the AlgoVM; 222 programs identical in CI.

Backends: Claude, OpenAI-compatible local models, offline mock, replay from traces.

Governance built into the language

Control	What it does	Syntax
Token, cost, energy budgets	checked before every call; INR and USD; Wh and CO_2	`budget cost 0.04 usd`
Forbidden attributes	a protected attribute in a goal or prompt does not compile	`forbid gender, caste`
Human review	by rule, by confidence, or always; a persistent review queue	`require human when confidence < 0.8`
Four-eyes approval	two different people must approve	`require approvals 2`
Tool permissions	only declared tools; untrusted tools are fenced	`tool web.fetch untrusted`
Organisation policy	org-wide caps that no program can loosen	`algo.config.json`
Risk levels	minimal · limited · high, each with required safeguards	`risk high "purpose"`

Trustworthy AI (v0.4)

Trustworthiness as evidence, not a claim

- **Risk levels** that change what the compiler demands.
- **Privacy**: `redact pii`; protected values withheld even with typos.
- **Explanations**: `explain with evidence` quotes inputs and tool results.
- **Untrusted tools** fenced against prompt injection; MCP servers pinned by SHA-256.
- **Signed audit**: Ed25519, and since v0.6 ML-DSA (FIPS 204) or hybrid post-quantum signatures.
- **Retention**: audit and trace retention with signed checkpoints.
- **System cards** and **trust reports** generated from the program itself.
- **Evidence packs**: signed zip for auditors, verifiable offline.

Uncertainty and evals

Quality, fairness and robustness tested like code

```
eval trust on main {
    case "salaried" (app = "APP-3101 ...") {
        expect status ok
        expect fair across gender [woman, man]
        expect robust (typos, paraphrase, reorder)
               consistency >= 0.9
        expect explained
    }
}
```

```
$ algo eval programs/loan_decision.algo
✓ salaried applicant
  fair across gender: same decision, gap 0.000
  robust to typos, paraphrase, reorder:
  consistency 1.00 over 6 variants
✓ self-employed applicant
  2/2 cases pass
```

`uncertain<T>` samples an agent several times; flows branch on `.confidence`; low confidence goes to a person.

Typed agent-to-agent messages (v1.0)

```
type Party { name: text, phone_verified: bool }
type Claim { policy: text, amount: number,
             fraud_risk: score, claimant: Party }

agent Intake(note: text) -> Claim {
    goal "Fill in every field of the claim in {note}"
}
flow main(note: text) {
    claim = Intake(note)
    if claim.amount < 50000 and claim.fraud_risk < 0.7 {
        emit FastTrack(claim)
    } else {
        emit Investigate(claim, claim.amount)
    }
}
```

- Fields and argument types checked by the compiler (E131, E132).
- Every model answer validated against the type at run time.
- The live model is told the exact JSON shape.
- PII masked field by field.

Beyond the core

Real-time events, Quantum AI, interoperability and the ecosystem

AlgoPulse™

Event streams in the language: topics, keys, dedupe, escalation and dead-letter topics, back-pressure.

Quantum AI

Simulator, noisy simulators, IBM, Braket and Azure backends; QPU budgets; VERIFIED / BOUNDED answers; VQE, kernels, QAOA.

Interoperability

Flows and agents as MCP servers; typed inputs; token streaming; cost and energy dashboard.

Ecosystem

AlgoIR + AlgoVM, signed packages and registry, Studio with SSO and roles, VS Code, Python SDK.

Quantum hardware backends are exercised in CI with stand-in SDKs; they need the provider's credentials to run on a real device.

No quantum speed-up is claimed: quantum answers are checked before use.

Comparison with general-purpose languages

What each language is designed for

Language	Paradigm	Typing	Concurrency	Strength
AlgoLang	agent-oriented, declarative flows	static: text, score, list, json, number, typed messages	`par`, `par for`, streams	governed AI decisions: policies, review, audit, evals
Python	multi-paradigm	dynamic (optional hints)	threads, asyncio, processes	ecosystem for AI, data and scripting
Java	object-oriented	static	threads, virtual threads	large enterprise systems on the JVM
C / C++	procedural / multi-paradigm	static	threads	performance, systems, embedded
JavaScript / TS	multi-paradigm	dynamic / static (TS)	event loop, async/await	web and full-stack applications
Rust	multi-paradigm, ownership	static	threads, async; data-race freedom	memory safety without a garbage collector
Go	procedural, CSP	static	goroutines and channels	cloud services and tooling

AlgoLang is a domain-specific language: it orchestrates AI agents and delegates computation to tools written in general-purpose languages.

Comparison with scientific, logic and AI-specific languages

Language	Focus	Core idea	Relation to AlgoLang
AlgoLang	governed AI agents	agents with policies; uncertainty and evals as language features	—
R	statistics	vectorised data analysis, rich statistical libraries	tools for analysis inside agents
Julia	scientific computing	multiple dispatch, JIT compilation	numerical tools inside agents
MATLAB	engineering maths	matrices, toolboxes	engineering tools inside agents
SQL	data querying	declarative relational queries	data access tools for agents
Prolog	logic programming	facts, rules, unification	symbolic rules beside model agents
Lisp	symbolic AI	code as data, macros	historic AI language; no governance layer
LMQL	constrained LLM prompting	queries with constraints on model output	prompt-level; AlgoLang is program-level
Q# / Qiskit	quantum programming	quantum circuits and algorithms	AlgoLang calls circuits as verified tools

Comparison with agent frameworks

Libraries on a host language vs. a language with guarantees

Framework	Form	Approach	Governance
AlgoLang	language + compiler	agents and flows as syntax; checked before running	built in: compile-time and run-time
LangChain / LangGraph	Python, JS library	chains and stateful agent graphs	added by the developer in code
CrewAI	Python library	role-based crews of agents	added by the developer in code
AutoGen	Python, .NET library	multi-agent conversations	added by the developer in code
Semantic Kernel	C#, Python, Java SDK	plugins and planners in enterprise apps	added by the developer in code
DSPy	Python library	declarative LM programs, optimised prompts	added by the developer in code

Frameworks are powerful and mature, and have far larger communities. AlgoLang's difference is **where** the guarantees live: in a compiler that refuses an unsafe program, instead of in conventions each team must remember to follow.

Feature matrix

✓ built in ◐ via libraries or conventions ✗ not available

Capability	AlgoLang	Python	Java	Rust	TS	Julia	Lang-Graph	CrewAI
Agents as a language construct	✓	✗	✗	✗	✗	✗	◐	◐
Budgets checked before running	✓	✗	✗	✗	✗	✗	◐	◐
Human review / four-eyes	✓	◐	◐	◐	◐	◐	◐	◐
Uncertainty as a type	✓	◐	◐	◐	◐	◐	✗	✗
Fairness and robustness evals	✓	◐	◐	✗	◐	◐	◐	◐
Signed, hash-chained audit	✓	◐	◐	◐	◐	◐	◐	◐
Typed agent messages	✓	◐	◐	◐	◐	◐	◐	◐
Quantum tools, verified	✓	◐	◐	◐	◐	◐	✗	✗
General-purpose computing	✗	✓	✓	✓	✓	✓	◐	◐
Mature ecosystem and community	✗	✓	✓	✓	✓	✓	✓	✓
Raw performance	◐	◐	✓	✓	◐	✓	◐	◐

The same guarantee, two ways

A budgeted agent with human review on low confidence

Python with a framework (sketch)

```python
def triage(claim):
    if tokens_used() > 2000:
        raise BudgetError()
    out = llm.invoke(prompt(claim))
    score, conf = parse(out)
    log_audit(claim, out)        # remember to sign
    if conf < 0.8:
        queue_for_review(out)    # remember to persist
    if "gender" in prompt(claim):
        ...                      # remember to check
    return score
```

AlgoLang

```
agent Triage(claim: text) -> uncertain<score> {
    goal   "Score {claim}"
    policy { budget tokens 2000
             require human when confidence < 0.8
             forbid gender }
}
```

Budget, review queue, forbidden attribute and signed audit are **enforced**, not remembered.

Where AlgoLang is not the right tool

An honest view

- **General computation.** No loops over numbers, no data structures beyond its types: write those as tools in Python or another language.
- **High-performance code.** The runtime is Python; speed comes from tools.
- **A young ecosystem.** One maintainer team, a small community and few third-party packages so far.

Use AlgoLang when

- AI agents make decisions that affect people or money
- you must show regulators who decided and why
- fairness, budgets and human review are requirements, not wishes
- you want agent programs that are reviewed and tested like code

22 governed enterprise applications

Each with policies, company tools, knowledge base, evals, behaviour tests and system cards

Enterprise

- Insurance claims operations
- Hospital care coordination
- Bank credit and KYC
- University admissions and OBE
- SOC incident command
- Supply chain control tower
- Citizen services
- Legal contract desk

AI, markets and quantum

- Algo trading desk
- Quantum optimization lab
- AlgoQ Quantum AI lab
- AlgoGraph LLM & SLM optimizer
- ML model factory
- RL control room
- Responsible XAI studio

Large-scale domains (v0.5)

- NLP document intelligence
- Bharat language desk (8 scripts)
- Digital health monitor (NEWS2)
- Network biology lab
- Predictive maintenance
- Smart city operations
- Transport network optimizer

22	120	222	34
applications	domain packs	programs on AlgoVM	compile checks

Toolchain

```
$ pip install algolang
$ algo check app.algo
$ algo run app.algo --arg "claim=..."
--stream
$ algo eval app.algo
$ algo card app.algo
$ algo build app.algo --ir --sign
$ algo mcp serve app.algo
$ algo pkg install insurance@^1.0
$ algo studio
```

- **CLI** `algo`: check, run, batch, eval, card, report, audit, evidence, build, pkg, mcp.
- **Studio**: browser IDE with review queue, dashboard, SSO and roles.
- **Python SDK** and Jupyter / Colab.
- **VS Code** extension and language server.
- **CI**: Linux, macOS and Windows on every change.

Roadmap delivered

Every planned release shipped, one at a time



Latest: 1.0 typed messages · AlgoIR and AlgoVM · signed packages · Studio SSO and roles.

After 1.0 AlgoVM for browser and edge · organisation registries · federated review · long-running agents.

Getting started in five minutes

```
$ git clone https://github.com/
  drssnaiml1-cell/algolang
$ cd algolang && pip install -e .
$ algo run examples/hello.algo \
  --arg "name=Asha"
$ algo studio
```

Runs offline with the mock model; add `ANTHROPIC_API_KEY` and `--live` for real models.

- **50 sample programs** with a PDF guide.
- **108-page language manual** (PDF).
- **Website**: playground and package registry.
- **Releases** with wheels for every version.

Collaborate with us

- **Researchers and Ph.D. scholars**: agentic AI, XAI, AI governance, quantum AI; joint papers and theses.
- **Universities**: teach governed AI with a language built for it.
- **Industry**: pilot trustworthy agents in a real workflow.
- **Developers**: contribute to an open-source language.

Open research questions

- Fairness guarantees at compile time
- Verifying quantum answers before agents act
- Auditable AI decisions in health and finance
- Typed messages and multi-agent reliability

Thank you

Build AI people can trust, then prove it.

Dr. S. Satyanarayana, Ph.D., PDF (AI)

Founder, CEO & Chief Agentic AI Scientist · AlgoProfessor AI R&D Solutions, Hyderabad

github.com/drssnaiml1-cell/algolang · www.algoprofessor.com · ceo@algoprofessor.com

AlgoLang is open source under the Apache 2.0 licence.